7-3 · Embedding과 Vector DB · 실습 1H

임베딩 생성 실습

차원을 세고, 유사도를 손으로 계산한다

수도권ICT 이노베이션스퀘어
데이터 엔지니어링 & RAG 파이프라인 160H · 3기

이 시간의 목표

1시간 뒤에는

① rag_env.py 의 get_emb() 로 임베딩 객체를 받는다

② embed_query 로 벡터를 만들고 차원(768 또는 1536)을 확인한다

③ cos 함수를 짜서 비슷한 문장과 다른 문장을 비교한다

④ 유사도 표 · 배치 속도 · 임베딩이 약한 곳을 실측으로 본다

여기서 메모한 차원 수를 다음 1.5H 실습의 컬렉션 size 에 그대로 씁니다

실습 흐름

여섯 단계 — 파일을 만들고 python 으로

단계마다 cat > 파일.py << 'EOF' … EOF → python 파일.py — 새로 설치할 것은 없습니다

1. 패키지	2. 차원	3. 유사도	4. 유사도 표	5·6. 배치·한계
pip list 다섯 줄	emb_dim.py 768	emb_cos.py 0.587 · 0.223	emb_table.py 5×5	emb_batch.py emb_limits.py

1단계

패키지를 확인하세요

1. **cd ~/data-eng/week7 → source .venv/bin/activate**
   Windows(WSL2) · macOS · 리눅스 모두 같다
2. **pip list | grep -i -E "qdrant|ollama|openai"**
   langchain-ollama 1.1.0 · langchain-openai 1.7.3 · ollama 0.6.3
3. **openai 3.42.1 · qdrant-client 1.20.0**
   다섯 줄이면 된다(실측) — 7-1 과 7-2 정리·회고에서 다 들어왔다

임베딩 객체

rag_env.py

get_emb()

> **OpenAI 키(.env)가 있으면 text-embedding-3-small(1536), 없으면 Ollama embeddinggemma(768)**

2단계

emb_dim.py — 문장 하나를 벡터로

```
cat > emb_dim.py << 'EOF'
from rag_env import PROVIDER, get_emb

emb = get_emb()
v1 = emb.embed_query("데이터 파이프라인을 매일 실행합니다")
print(PROVIDER, "차원:", len(v1))
print("앞 5개 값:", [round(x, 4) for x in v1[:5]])
EOF
python emb_dim.py
```

실측 ollama 차원: 768 · 앞 5개 값: [-0.097, 0.0077, ...] · 첫 호출 약 40초 · OpenAI 는 1536 — 이 수를 메모

3단계

emb_cos.py — 코사인 유사도를 직접 계산

1. **v1 「데이터 파이프라인을 매일 실행합니다」**
   기준 문장

2. **v2 「배치 작업을 하루에 한 번 돌립니다」**
   겹치는 낱말이 거의 없다 → 비슷한 문장: 0.587

3. **v3 「오늘 점심은 김치찌개입니다」**
   다른 문장: 0.223

키워드 검색이라면

v2 도

0점

> **cos(a, b) = dot / (na × nb) · 실측 embeddinggemma — 값의 크기는 모델마다 다르니 두 값의 차이를 본다**

4단계

emb_table.py — 문장 5개의 유사도 표

번호	문장	실측
0	ETL은 추출, 변환, 적재 순서로…	ELT(1)와 0.65 — 가장 가깝다
1	ELT는 먼저 적재하고 나중에 변환	ETL(0)과 0.65
2	Airflow는 작업 순서를 관리하는 도구	ETL(0)과 0.39 — Kafka 보다 ETL 쪽
3	Kafka는 이벤트를 실시간으로…	—
4	오늘 날씨가 참 좋습니다	나머지와 0.16~0.26 — 가장 멀다

대각선은 1.00(자기 자신) · Airflow 가 ETL 쪽에 붙은 까닭은 둘 다 파이프라인 이야기 — 예상과 다른 쌍을 채팅에 적어 보세요

5단계

emb_batch.py — 하나씩 vs 한꺼번에

하나씩 — embed_query 다섯 번

실측 12.37초 (Ollama)

- for s in sents: emb.embed_query(s)
- 부를 때마다 호출이 한 번씩
- 인터넷 너머 API 면 왕복이 매번 붙는다

VS

배치 — embed_documents 한 번

실측 10.06초 (Ollama)

- emb.embed_documents(sents)
- 내 PC 안이라 차이가 작게 나왔다
- 문서 수천 개면 몇 분과 몇 시간 차이

6단계 — 일부러 실패시키기

emb_limits.py — 반대말과 식별자

1. **「성공했습니다」 vs 「실패했습니다」**
   0.741 — 뜻이 반대인데 3단계 「비슷한 문장」(0.587)보다 가깝다
2. **「주문번호 A-1001」 vs 「A-1002」**
   0.862 — 다른 주문번호 둘이 거의 같은 벡터
3. **왜 그런가**
   문맥이 거의 같기 때문
4. **그래서**
   정확한 식별자는 벡터 검색에 맡기지 않는다 — 7-5 에서 BM25 를 함께

> **이론의 비교표를 기억하세요 — 정확한 코드·품번은 키워드 검색이 강하다**

확인 포인트

이 1시간이 끝날 때 확인할 것

1. pip list 에 다섯 줄 — qdrant-client 1.20.0 포함
2. ollama 차원: 768(OpenAI 1536) — 메모해 두었다
3. 비슷한 문장 0.587 · 다른 문장 0.223 — 차이가 뚜렷하다
4. 대각선 1.00 · ETL·ELT 0.65 · 날씨 문장이 가장 멀다
5. 하나씩 12.37초 / 배치 10.06초 — embed_documents 를 쓴다
6. 성공 vs 실패 0.741 · A-1001 vs A-1002 0.862

수치는 embeddinggemma 실측 — OpenAI 경로는 값이 다르니 두 값의 차이를 봅니다

7-3 · 실습 1H 마무리

다음은 Qdrant 실습 (1.5H)

Qdrant 를 도커로 띄우고 컬렉션을 만들어 벡터를 적재하고, 낱말이 겹치지 않는 질문으로 검색합니다

- ✓ **docker run 으로 qdrant v1.20.0 기동**
- ✓ **size·distance 로 컬렉션 생성**
- ✓ **차원을 일부러 틀려 오류 보기**
- ✓ **query_points 로 의미 검색·필터**

가져갈 것

rag_env.py · 메모한 차원 수